Python Practical Programs for Mathematics / Statistics / ML Lab

1. Matrix Operations using NumPy

Dataset: Random or user-defined matrices

```
import numpy as np

A = np.array([[1,2],
              [3,4]])

B = np.array([[5,6],
              [7,8]])

print("Addition:")
print(A + B)

print("Multiplication:")
print(np.dot(A, B))

print("Determinant of A:")
print(np.linalg.det(A))

print("Inverse of A:")
print(np.linalg.inv(A))

eigenvalues, eigenvectors = np.linalg.eig(A)

print("Eigenvalues:")
print(eigenvalues)

print("Eigenvectors:")
print(eigenvectors)
```

2. Principal Component Analysis (PCA)

Dataset: 2D sample dataset

```
import numpy as np
from sklearn.decomposition import PCA

X = np.array([[2,3],
              [3,4],
              [4,5],
              [5,6]])

pca = PCA(n_components=1)

reduced = pca.fit_transform(X)

print("Reduced Data:")
print(reduced)
```

3. Naive Bayes Spam Classifier

Dataset: Simple spam/ham messages

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
messages = [
```

```
    "Win money now",
    "Claim your prize",
    "Hello friend",
    "Meeting tomorrow"
]

labels = ["spam", "spam", "ham", "ham"]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(messages)

model = MultinomialNB()

model.fit(X, labels)

test = vectorizer.transform(["Win prize now"])

prediction = model.predict(test)

print("Prediction:", prediction[0])
```

4. Coin Toss & Poisson Simulation

Dataset: Generated random values

```
import numpy as np
import matplotlib.pyplot as plt

tosses = np.random.choice(['H','T'], size=100)

heads = np.sum(tosses == 'H')
tails = np.sum(tosses == 'T')

print("Heads:", heads)
print("Tails:", tails)

plt.bar(['Heads','Tails'], [heads, tails])

plt.title("Coin Toss Simulation")

plt.show()

data = np.random.poisson(lam=5, size=1000)

print("Mean:", np.mean(data))

plt.hist(data)

plt.title("Poisson Distribution")

plt.show()
```

5. Dice Roll & Binomial Simulation

Dataset: Generated random values

```
import numpy as np
import matplotlib.pyplot as plt

rolls = np.random.randint(1,7,size=100)

counts = []

for i in range(1,7):
    count = np.sum(rolls == i)
    counts.append(count)

    print("Count of", i, "=", count)
```

```
plt.bar([1,2,3,4,5,6], counts)

plt.title("Dice Roll Simulation")

plt.show()

data = np.random.binomial(n=10, p=0.5, size=1000)

print("Mean:", np.mean(data))

plt.hist(data)

plt.title("Binomial Distribution")

plt.show()
```

6. Sampling Techniques using Python

Dataset: Hospital waiting time data

```
import pandas as pd

data = pd.DataFrame({
    'Waiting_Time':[10,15,20,25,30,35,40,45]
})

sample = data.sample(n=4, random_state=1)

print("Random Sample:")
print(sample)
```

7. Hypothesis Testing (Z-Test for Mean)

Dataset: Hospital waiting times

```
import numpy as np

sample_mean = 52
population_mean = 50
sigma = 5
n = 100

z = (sample_mean - population_mean) / (sigma / np.sqrt(n))

print("Z value =", z)

if abs(z) > 1.96:
    print("Reject Null Hypothesis")
else:
    print("Fail to Reject Null Hypothesis")
```

8. Hypothesis Testing (t-Test)

Dataset: Push-up performance data

```
from scipy import stats

before = [20,22,19,24,21]
after = [25,27,23,28,26]

t, p = stats.ttest_rel(after, before)

print("t value =", t)
print("p value =", p)
```

```
if p < 0.05:
    print("Training Improved Performance")
else:
    print("No Significant Improvement")
```

9. Hypothesis Testing (Z-Test for Proportion)

Dataset: Customer satisfaction survey

```
import numpy as np

p_hat = 0.58
p = 0.5
n = 200

z = (p_hat - p) / np.sqrt((p*(1-p))/n)

print("Z value =", z)

if abs(z) > 1.96:
    print("Reject Null Hypothesis")
else:
    print("Fail to Reject Null Hypothesis")
```

10. Chi-Square Test of Independence

Dataset: Department vs learning mode

```
import numpy as np
from scipy.stats import chi2_contingency

data = np.array([
    [30,20],
    [25,35]
])

chi2, p, dof, expected = chi2_contingency(data)

print("Chi-Square =", chi2)
print("p value =", p)

if p < 0.05:
    print("Variables are dependent")
else:
    print("Variables are independent")
```

11. Simple Linear Regression (Food Delivery Revenue)

Dataset: Orders vs revenue

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

orders = np.array([10,20,30,40,50]).reshape(-1,1)

revenue = np.array([1000,2200,3600,4000,5000])

model = LinearRegression()

model.fit(orders, revenue)

predicted = model.predict(orders)
print("Slope:", model.coef_[0])
```

```
print("Intercept:", model.intercept_)

plt.scatter(orders, revenue)

plt.plot(orders, predicted)

plt.title("Orders vs Revenue")

plt.show()
```

12. Simple Linear Regression (Rainfall vs Water Level)

Dataset: Rainfall and reservoir level

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

rainfall = np.array([10,20,30,40,50,60,70,80,90,100]).reshape(-1,1)

water_level = np.array([2,4,5,7,8,10,11,13,14,16])

model = LinearRegression()

model.fit(rainfall, water_level)

predicted = model.predict(rainfall)

plt.scatter(rainfall, water_level)

plt.plot(rainfall, predicted)

plt.title("Rainfall vs Water Level")

plt.show()
```

13. Directional Derivative using Python

Dataset: Function based calculation

```
import numpy as np

gradient = np.array([2, 3])

direction = np.array([1, 1])

unit_direction = direction / np.linalg.norm(direction)

directional_derivative = np.dot(gradient, unit_direction)

print("Directional Derivative =", directional_derivative)
```

14. Gradient Descent Algorithm

Dataset: Quadratic loss function

```
import numpy as np
import matplotlib.pyplot as plt

x = 10
learning_rate = 0.1

values = []
for i in range(50):
```

```
    gradient = 2*x
    x = x - learning_rate * gradient
    values.append(x)
print("Minimum value near:", x)
plt.plot(values)
plt.title("Gradient Descent Convergence")
plt.show()
```